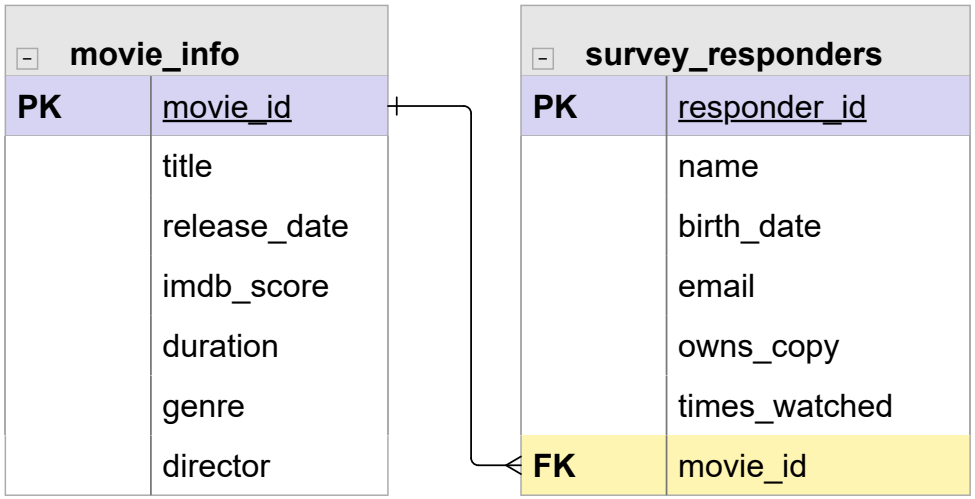


James' Super Cool IS 201 Cheat Sheet

Index

	Slide	Topic
Unit 2	2	ERDs
	6	SQL
Unit 3	12	Flow Charts
	13	VBA
Unit 4	19	Statistics
	20	Tableau
	21	Solver
Unit 5	22	HTML
	31	CSS

ERD: Primary / Foreign Keys



movie_id	title
1	Monty Python…
2	Count of Mon…
3	Princess Bri…

responder_id	name	movie_id
1	James	1
2	Gaskin	2
3	Megan	1

Want the duration of James's favorite movie? Look up his `movie_id` (1) in `movie_info`. The movie's info is stored once there, not copied onto James's and Megan's rows. Avoiding that repetition is the power of foreign keys!

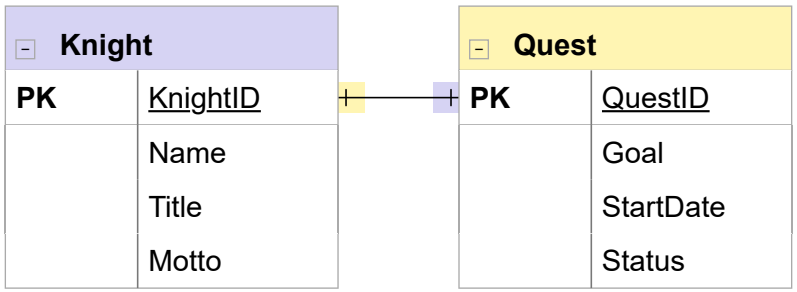
A primary key is a unique identifier (think KnightID or MovieID)

A foreign key is another table's primary key, used to link two tables together

Entity
Relationship
Diagram

ERD: Cardinality

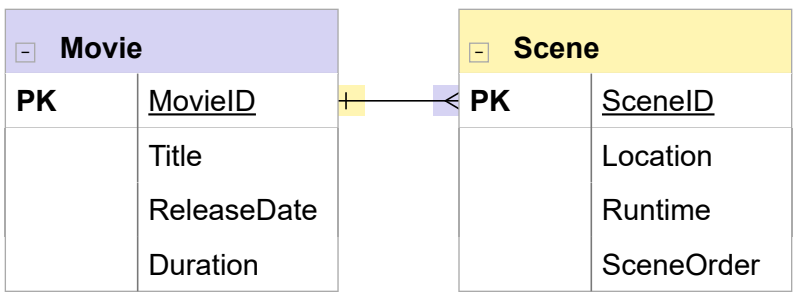
Read left-to-right AND right-to-left



One to One (1:1)

A knight has one quest

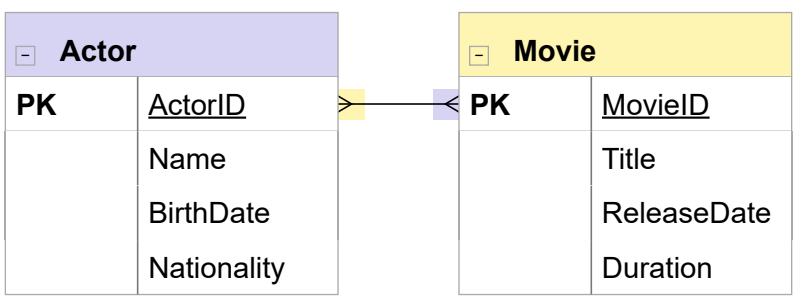
A quest belongs to one knight



One to Many (1:N)

A movie has multiple scenes

A scene belongs to one movie



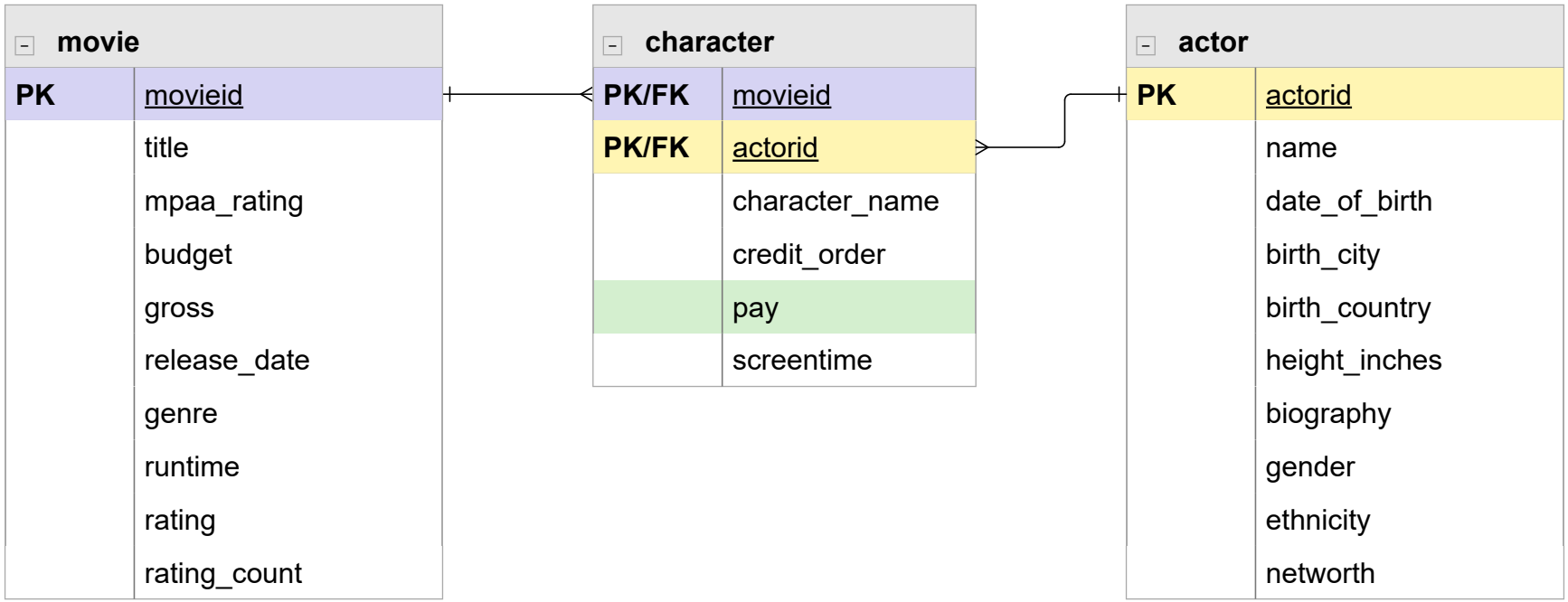
Many to Many (M:N)

An actor can be in many movies

A movie can have lots of actors

Note! M:N needs a composite table (next slide)

ERD: Associative Table

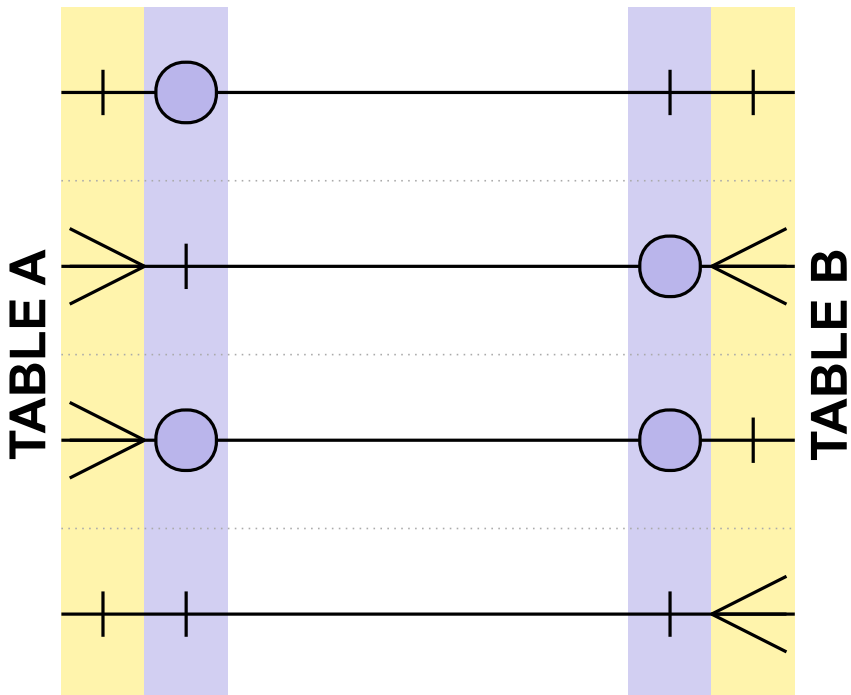


To know how much Graham Chapman got paid to play King Arthur in *Monty Python and the Holy Grail*, you need both the movie and the actor. Neither alone is enough: King Arthur has also been played by Sean Connery (*First Knight*) and Clive Owen (*King Arthur*), and Chapman's pay was probably different when he played Brian in *Life of Brian*.

ERD: Minimum / Maximum Cardinality

The inner marks are minimum (0 or 1)

and the outer marks are maximum (1 or many)



A could have 1 B or 1 B

B could have 0 A or 1 A

A could have 0 B or many B

B could have 1 A or many A

A could have 0 B or 1 B

B could have 0 A or many A

A could have 1 B or many B

B could have 1 A or 1 A

Think: a knight could have 0 horses (coconuts!) or multiple.

go to gaskination.com/sql

SQL: Clauses

Always written in this order. Only `SELECT` *and* `FROM` *are required.*

```
SELECT      -- which attributes to show (* = all, DISTINCT = no duplicate rows)
FROM        -- the table to pull from (tableA)
JOIN        -- tableB ON tableA.key = tableB.key   see Join slides
WHERE       -- keep only rows that pass a filter   see Where slides
    AND/OR  -- add more filters (only write WHERE once)
GROUP BY    -- with aggregates: list every SELECT attribute that isn't aggregated
HAVING      -- like WHERE, but filters groups
ORDER BY    -- sort by an attribute: ASC (default, low to high) or DESC
LIMIT       -- only show the first N rows (e.g. LIMIT 10)
```

```
-- aggregates turn many rows into one value
-- function(attribute) AS new_name
SELECT avg(imdb_score) AS avg_score
-- examples: count(), sum(), avg(), min(), max()

-- you can also do math with attributes (+ - * /)
SELECT imdb_score / duration AS score_per_min
```

Structured
Query
Language

SQL: Where (1/2)

Text

in single quotes

```
name = 'James'
```
→ exactly 'James'

```
name != 'James'
name <> 'James'
```
→ anything but 'James'

```
title LIKE '%Mont%'
```
→ finds both 'Monty' and 'Monte'

% = any number of characters
_ = exactly one character

```
name IN ('James', 'Gaskin', 'Reese')
```
→ exact match for any of these

Blank

any data type

```
movie_id IS NULL
```
→ blank / missing

```
movie_id IS NOT NULL
```
→ has data

SQL: Where (2/2)

Numbers

no quotes

```
imdb_score > 8
```

```
imdb_score < 8
```

→ greater than / less than

```
imdb_score >= 8
```

```
imdb_score <= 8
```

→ … or equal to

```
duration BETWEEN 90 AND 120
```

→ 90 to 120 minutes (includes both)

Booleans

no quotes

```
owns_copy = TRUE
```

→ people who own the movie

```
owns_copy = FALSE
```

→ everyone else

Dates

single-quoted

'YYYY-MM-DD'

```
release_date < '1975-01-01'
```

→ released before 1975, the *Holy Grail* year

```
birth_date < '1940-01-01'
```

→ older fan born before 1940

```
release_date BETWEEN '1975-01-01' AND '1975-12-31'
```

→ released in the year *Monty Python and the Holy Grail* premiered

SQL: Join (1/2)

JOIN *combines two tables, keeping rows that match in both*

```
SELECT survey_responders.name, movie_info.title
FROM survey_responders
JOIN movie_info
  ON survey_responders.movie_id = movie_info.movie_id
```

`ON` says which columns have to match. Each `survey_responders` row is paired with the `movie_info` row that has the same `movie_id`.

survey_responders

name	movie_id
James	1
Gaskin	2
Reese	4

movie_info

movie_id	title
1	Monty Python…
2	Count of Mon…
3	Princess Bri…

→

result

name	title
James	Monty Python…
Gaskin	Count of Mon…

Reese's movie (4) isn't in movie_info, and no one picked Princess Bride (3), so both are left out.

SQL: Join (2/2)

A

responder_id	name	movie_id
1	James	1
2	Gaskin	2
3	Reese	4

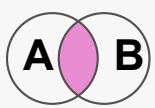
B

movie_id	title
1	Monty Python…
2	Count of Mon…
3	Princess Bri…

```
SELECT name, title
FROM A
_____ JOIN B
    ON A.movie_id = B.movie_id
```

INNER JOIN

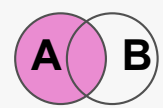
only matches (same as just JOIN)



name	title
James	Monty Python and the Holy G…
Gaskin	Count of Monte Cristo

LEFT JOIN

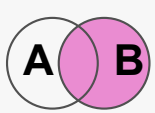
all of A + matches from B



name	title
James	Monty Python and the Holy G…
Gaskin	Count of Monte Cristo
Reese	*NULL*

RIGHT JOIN

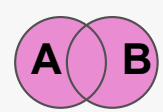
all of B + matches from A



name	title
James	Monty Python and the Holy G…
Gaskin	Count of Monte Cristo
NULL	Princess Bride

FULL OUTER JOIN

everything from both



name	title
James	Monty Python and the Holy G…
Gaskin	Count of Monte Cristo
Reese	*NULL*
NULL	Princess Bride

SQL: Example

Question: *For well-rated (7+) movies with more than one fan: how many years after each movie came out were its fans born, on average?*

```
SELECT title,
       count(*) AS fans,
       avg((julianday(birth_date) - julianday(release_date)) / 365.25) AS avg_years_between
FROM survey_responders
JOIN movie_info ON survey_responders.movie_id = movie_info.movie_id
WHERE imdb_score >= 7
  AND birth_date IS NOT NULL
GROUP BY movie_info.movie_id, title
HAVING count(*) > 1
ORDER BY avg_years_between DESC
LIMIT 5
```

Flow Charts

The shapes

start/end

you can only have **ONE** start but you **CAN** have multiple ends

process (happens behind the scenes)

decision (T/F or Y/N)

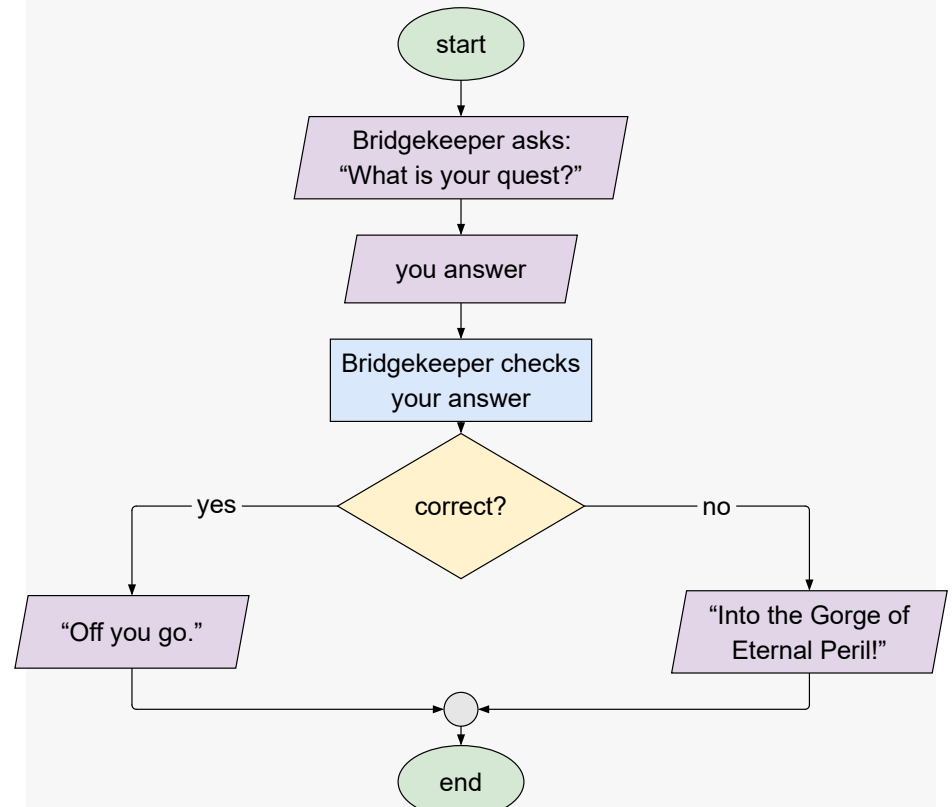
decisions are the **ONLY** thing that can have more than one arrow pointing **OUT** of them

input/output (what the user enters or sees)

connector

connectors are the **ONLY** thing that can have more than one arrow pointing **INTO** them

Example: the Bridge of Death



You only see and hear the input/output steps; the process happens in the Bridgekeeper's head.

Make yours at draw.io → when done: **File → Export as → PDF**

VBA: Basics

```
Option Explicit 'This makes it so that you can only use variables you've declared (VERY RECOMMENDED)

Sub thisIsMySubName()

    'BASICS!
```

1. object type	2. specify	3. what to do
Sheets	("Sheet1")	.Activate
Range	("A1:B3")	.Copy

```
    ' "Sheets" and "Worksheets" work the same:
    Sheets("MySheet").Delete
    Sheets.Add.Name = "MySheet"
    Worksheets("MySheet").Activate

    ' "Range" and "Cells" are equivalent for single cells:
    ' Range("E7") = Cells(7, 5) --> (Row, Column)
    Range("A1").Activate
    ActiveCell.Value = "We want... a shrubbery!"
         ' ActiveCell looks at whatever cell was last activated

    Range("A1:D3").Copy
    Range("E5").PasteSpecial

    ' Columns and Rows are what they sound like
    Columns("B:D").Delete
```

Visual
Basic for
Applications

VBA: Variables and Text

```
'DECLARE VARIABLES!
    Dim knights As Integer    'a whole number (use Long for big ones, like row counts)
    Dim price As Double       'a number with a decimal
    Dim quote As String       'a line of text
    Dim isBrave As Boolean    'True or False
    Dim ws As Worksheet       'an object (a sheet)

    knights = 5
    price = 12.99
    quote = "Tis but a scratch!"
    isBrave = True
    Set ws = Sheets("Sheet1") 'objects (sheets, workbooks, ranges) use Set instead of just =

'WORKING WITH TEXT!

    Dim myName As String, myOutput As String
    myName = "James"

    ' Combine strings with ampersands (&) -- don't forget spaces
    myOutput = "My name is " & myName & " and I seek the Grail!"

    ' Take characters from the left or right end
    myOutput = Left(myName, 1)     '"J"
    myOutput = Right(myName, 3)    '"mes"

    ' Change the case
    myOutput = UCase(myName)       '"JAMES"
    myOutput = LCase(myName)       '"james"
```

VBA: Navigation and Misc.

```
'NAVIGATION!

    ' Jump to the edge of the data (like Ctrl + arrow key)
    ActiveCell.End(xlDown).Activate
    ActiveCell.End(xlToLeft).Activate

    ' Offset(rows down, columns right) from a cell
    ActiveCell.Offset(1, 2).Value = "Ni!"   '1 down, 2 right of the active cell
    Range("A1").Offset(3, 4).Value = "Ni!"  '3 down, 4 right of A1 = E4

' MISCELLANEOUS!

    ' Bolding
    Range("A1").Font.Bold = True

    ' Autofitting columns
    Columns.AutoFit 'all columns
    Columns("A:C").AutoFit 'specific columns

    ' Currency format
    Range("A1").Style = "Currency"

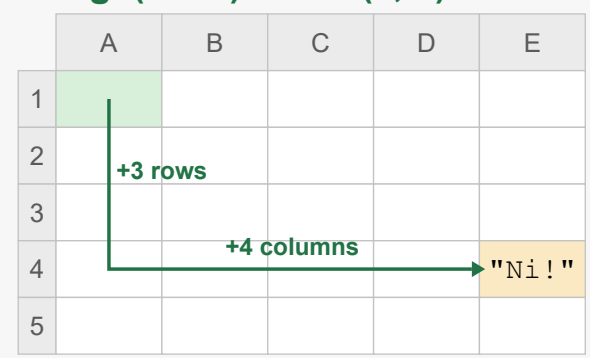
    ' Bottom border
    Range("A1").Borders(xlEdgeBottom).LineStyle = xlContinuous

'USER INTERFACE!

    ' Tell the user something (a pop-up box)
    MsgBox "Halt! Who would cross the Bridge of Death?"

    ' Ask the user to type something in
    Dim username As String
    username = InputBox("What is your name?")
```

Range("A1").Offset(3, 4)



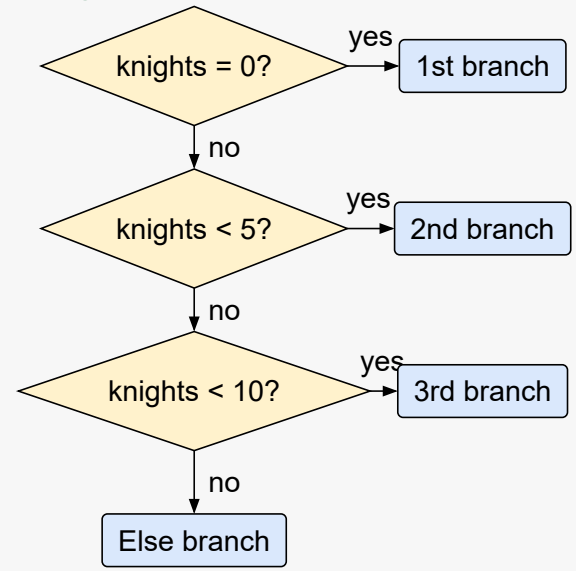
VBA: Conditionals

```
'CONDITIONALS!
    ' If Statements
    If knights = 0 Then
        ' your code here
    ElseIf knights < 5 Then
        ' your code here
    ElseIf knights < 10 Then
        ' your code here
    Else
        ' your code here (everything else)
    End If

    ' Case Statements: compare ONE value to several options
    ' (assume fruit, color, imdb_score, verdict are declared)
    Select Case fruit
        Case "apple", "strawberry"
            color = "red"
        Case "coconut"
            color = "brown"
        Case Else
            color = "unknown"
    End Select

    ' Another Case Statement (ranges of numbers)
    Select Case imdb_score
        Case Is >= 8
            verdict = "must watch (like Monty Python)"
        Case 6 To 8 'an 8 already matched above: first match wins
            verdict = "pretty good"
        Case Is < 6
            verdict = "I'm probably skipping this one"
    End Select
```

Only the first true branch runs



VBA: Loops

```
'LOOPS!

    ' For Loop
    Dim i As Integer
    For i = 0 To 5   'counts 0 to 5 (runs 6 times)
        ' your code here
    Next i

    ' For Each Loop: once per cell, from A2 down to the last filled row
    Dim x As Range, lastrow As Long
    lastrow = Cells(Rows.Count, "A").End(xlUp).Row 'last filled row in A
    For Each x In Range("A2", Cells(lastrow, "A"))
        ' your code here
    Next

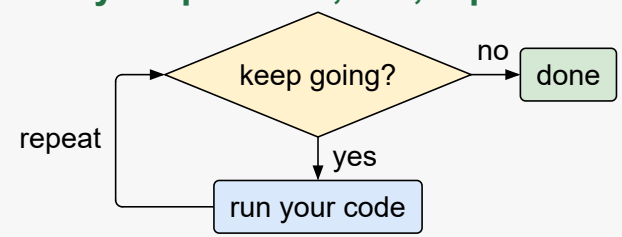
    ' For Each Loop 2: once per cell in a set range
    Dim myRange As Range
    Set myRange = Range("A1:E5")
    For Each x In myRange
        ' your code here
    Next

    ' Do While Loop: repeats WHILE the condition is true
    i = 0
    Do While i < 5 'condition does NOT have to be numbers
        ' your code here
        i = i + 1 'change something, or it loops forever
    Loop

    ' Do Until Loop: repeats UNTIL the condition becomes true
    Do Until i = 10 'condition does NOT have to be numbers
        ' your code here
        i = i + 1
    Loop

End Sub
```

Every loop: check, run, repeat



VBA: Functions

```
Function knightTitle(name As String, isBrave As Boolean) As String

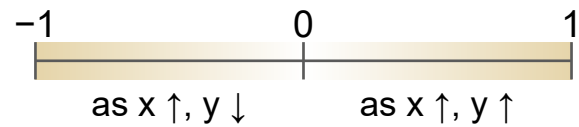
    ' a Function RETURNS a value you can use in a cell (a Sub just DOES things)
    ' the INPUTS go in the parentheses, each with a type
    ' the OUTPUT's type goes at the end (As String)
    ' so the function declaration works like Dims you get for free:
    '     Dim knightTitle As String
    '     Dim name As String
    '     Dim isBrave As Boolean

    ' use the inputs to decide the output...
    If isBrave Then
        knightTitle = "Sir " & name & " the Brave"
    Else
        knightTitle = "Sir " & name & " the Not-Quite-So-Brave"
    End If
    ' ...and return it by setting the FUNCTION'S NAME equal to it

End Function

' to use it in a cell:
' =knightTitle("Robin", FALSE)   gives   Sir Robin the Not-Quite-So-Brave
```

Statistics

TEST & USE CASE	OUTPUT	HOW TO READ IT
Correlation numeric – numeric (Pearson *r* coefficient)	→ r	 *strength = distance from 0; sign = direction*
Regression numeric – multi. numeric	→ R^2	 70% of the variance in y is explained by the x's
ANOVA numeric – mult. categorical **T-test** numeric – 2 pair categorical	→ p	usually 0.05  reject the null **significant** fail to reject the null **not significant** *p works like a percentage:* *0.05 = 5% to see this or more extreme*

Null Hypothesis

H_0: No x's are significant

Alt Hypothesis

H_A: At least one x is significant

Scientific Notation

aE**b** = a × 10^{b}

aE**3** = a × 1000 (big)

aE**−3** = a × 0.001 (small)

Tableau

“Default” means leave it alone

If the assignment says “default,” you don’t have to change anything. It’s the setting Tableau starts with.

Toolbar buttons worth knowing



Swap rows and columns: flips the x- and y-axis. A sideways chart can tell a clearer story.



Sort ascending: smallest to largest



Sort descending: largest to smallest

Solver

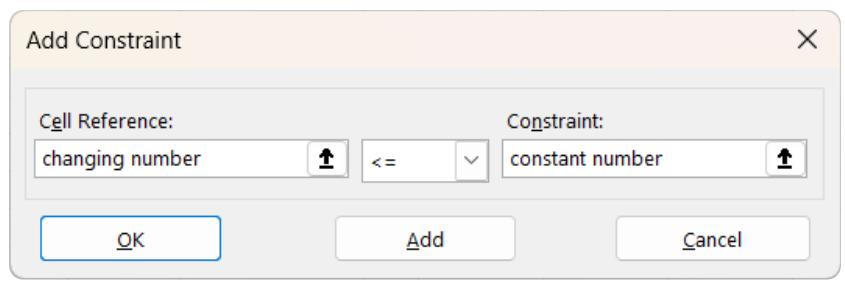
Two important Excel formulas:

=sum(A1:A5) → A1 + A2 + A3 + A4 + A5

=sumproduct(A1:A5, B11:B15) → A1*B11 + A2*B12 + … + A5*B15

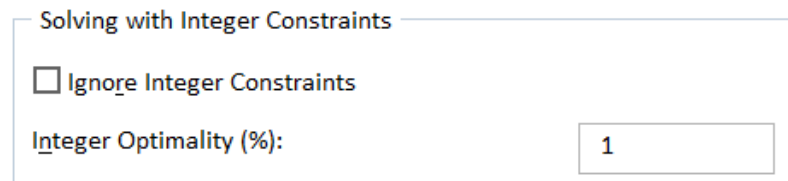
the two ranges should be the same size and shape

Constraints:



If integer isn't working:

Solver > Options > uncheck this box:



HTML: Setup

In an html file inside VS Code,

type an exclamation point and enter to automatically generate:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Monty Python and the Holy Grail</title>
</head>
<body>

</body>
</html>
```

Go here for free bootstrap themes:

startbootstrap.com/themes/portfolio-resume

Hyper**T**ext

Markup

Language

HTML: Tags

```
<!-- This is a comment -->
<h1>The Holy Grail</h1>
<h2>The Knights</h2>
<h3>Sir Robin</h3>

<p>Bravely ran away.</p>

<!-- Line Break -->
<br>
```

```
<!-- Unordered List (Bullets) -->
<ul>
    <li>Sir Lancelot</li>
    <li>Sir Galahad</li>
    <li>Sir Robin</li>
</ul>

<!-- Change <ul> to <ol>
for an ordered list (Numbers) -->
```

Every tag follows this pattern:

```
<tag>
  The Holy Grail
</tag>
```

← start tag
← what the tag applies to
← end tag: same name, plus a /

The start and end tags act like parentheses.

(a few, like `<br>`, have no end tag)

HTML: Anchors

```
<!-- Hyperlinks -->
<a href="camelot.html">Visit Camelot</a>

<!-- Folder References -->
<a href="grail-quest.html">Visit the Grail Quest</a>
<a href="cast/knights.html">Meet the Knights</a>
<a href="../camelot.html">Return to Camelot</a>   ← ../ goes up one folder

<!-- On-page Anchors -->
<h1 id="top">The Holy Grail</h1>
<a href="#top">Return to the Grail</a>   ← # + the id jumps to it on the same page
```

HTML: Images

```
<!-- Images -->
<img src="images/unladen-swallow.jpg" alt="An unladen swallow.">

<img src="images/african-swallow.jpg"
    alt="An African swallow carrying a coconut.">
```

Notice: images do NOT have an end tag </img>

(neither do line breaks </br>)

HTML: Divisions

Divs don't inherently do anything: they're invisible boxes you put content into so that you can isolate it for positioning and CSS styling using classes.

HTML:

```
<div class="coconut-square">
    <img src="assets/coconuts.jpg" alt="Two coconut halves">
</div>
```

CSS:

```
.coconut-square {
    width: 200px;
    height: 200px;
}
```

For more, see:

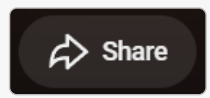
CSS: Selectors/Properties/Values

HTML: Embedding

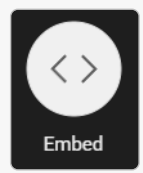
Embeds sometimes don't show up until your site is live

YouTube

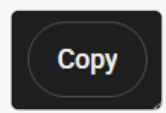
1. Under the video, click **Share**



2. Click **Embed**



3. Click **Copy**



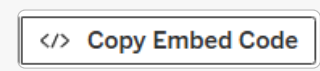
4. Paste it into your HTML

Tableau

1. Go to public.tableau.com/app/discover
2. Search for a relevant viz
3. Click the share icon

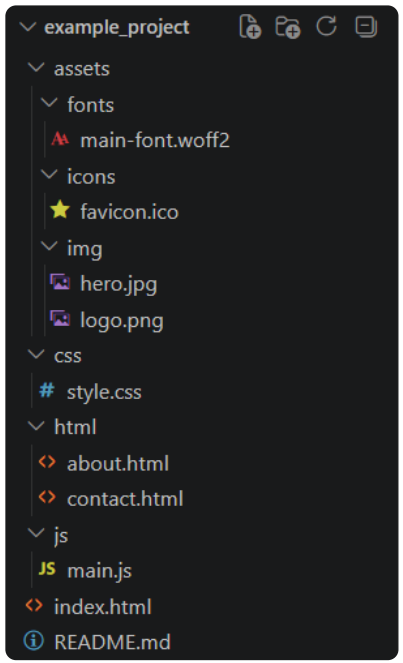


4. Click **Copy Embed Code**



5. Paste it into your HTML

HTML: Best Practices



Use folders!

Give each type of file its own folder: `css/`, `html/`, `js/`, and `assets/` (with `img/`, `fonts/`, `icons/` inside). `index.html` stays at the top level. It's your homepage, so it's the first page the site loads.

Never use spaces in file or folder names

Use a dash or underscore instead:

✓ `holy-grail.html` ✗ `holy grail.html`

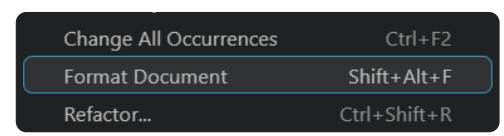
✓ `grail_project/` ✗ `grail project/`

Format your code

Fixes indentation and long lines.

Right click → **Format Document**

shortcut: `Shift+Alt+F`



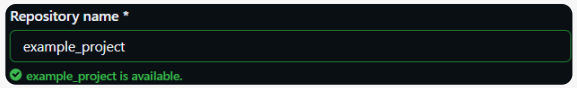
HTML: Uploading to GitHub

A. Create the repository

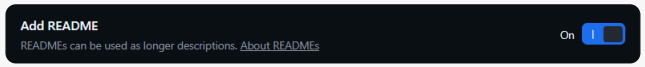
1. Go to github.com and sign in (or make a free account)
2. Click **New**



3. Type a **Repository name** (no spaces!)



4. Leave it **Public** and turn **Add README** on

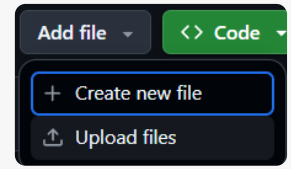


5. Click **Create repository** at the bottom

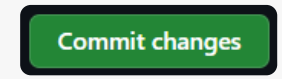


B. Upload your website

6. On your new repository's page, click **Add file → Upload files**



7. Open your project folder and drag everything *inside* it (`index.html`, `css/`, …) onto the page
   Upload the CONTENTS of your project, not the folder itself
8. Click **Commit changes** ("commit" = save)

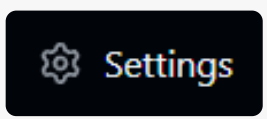


Next: turn it into a live website → Going Live

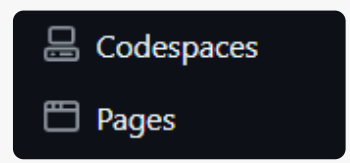
HTML: Going Live

On your repository's page (from the last slide):

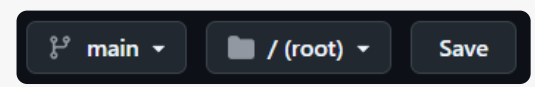
1. Click the **Settings** tab at the top



2. In the left sidebar, click **Pages**



3. Under **Branch**, change `None` to `main`, then click **Save**



4. Wait a couple of minutes, then **refresh** the page. Your website's link will appear at the top:



CSS: Basics

1. Connect it to your HTML

```
<head>
    <link rel="stylesheet" href="css/style.css">
</head>
```

Goes inside the `<head>` of every page. `href` is the path to your `.css` file.

2. Every rule looks like this

```
selector {
    property: value;
    property2: value2;
}
```

property = what to change, **value** = what to change it to. That's all CSS is! (see next slide)

3. It's cascading: the later rule wins

```
p {
    color: blue;
}
p {
    color: red;
}
```

If Sir Lancelot paints Camelot's gate blue, then repaints it red, it ends up red. Later CSS rules work the same way.

Cascading
Style
Sheets

CSS: Selectors / Properties / Values

Built-in selectors

Use a tag name (`body`, `h1`, `p`, `a`, `img`, …) to style every one of those tags.

```
body {   ← every <body>
    background-color: #f0f0f0;
    font-family: Arial, sans-serif;
    margin: 0;
    padding: 20px;
}
```

Class (custom) selectors

Make your own: a **period** + any name. Styles only tags with that class.

```
.grail-worthy {
    background-color: yellow;
    font-weight: bold;
}
```

Apply it in your HTML (no period):

```
<p class="grail-worthy">Ni!</p>
```

There are A LOT of selectors and properties. Search for the CSS you need to style your Grail quest site!

jimna-h.github.io/james_super_cool_is_201_cheatsheet



Suggestions? Reach out to me at jbruce1@byu.edu

IS 201 TA Lab